

MAD I - Project Report

Student Details

Name: P Vithyasagar

Roll Number: 24F2001900

Email: 24f2001900@ds.study.iitm.ac.in

Project Details

Project Title: Hospital Management System (HMS) Web App

Problem Statement: To build a web application that allows **Admins, Doctors, and Patients** to interact with the system based on their roles.

Approach:

The app is built using **Flask** for the backend to handle routing and database logic. It utilizes **Role-Based Access Control** to distinguish between Admins (who manage doctors) and Doctors (who manage patients). The frontend uses **Jinja2** and **Bootstrap** for a responsive interface to view patient history and appointments.

This project is a Hospital Management System (HMS) built using Flask.

It enables admin, doctor, and patient workflows, including user management, appointment scheduling, blacklist handling, and dashboard-based access.

AI/LLM usage

Component / Tool	Purpose / Role in Project	Usage %	Notes
Flask (Routes/Logic)	Backend logic, routing, request handling, simple REST APIs	25%	For standard CRUD routes, blueprint setup, and basic controller functions.
SQLite + SQLAlchemy	Data modeling, relationships, persistence, schema definition	20%	Drafting model classes, defining relationships, and generating migration scripts.
HTML + Jinja2	Server-side template rendering, basic page structure	20%	Form structures, and complex loops.
Bootstrap / CSS	Styling, responsive layout, component integration	5%	Low extent for getting correct class names or initial custom theme suggestions.
Authentication (Auth/Roles)	Login/logout, session management, role-based access control	25%	Security implementations (hashing, session setup, decorators)
Admin CRUD	Core Create, Read, Update, Delete views for system objects (doctors, patients)	30%	Usage for generating the repetitive template and route logic for standard admin tables and forms.

Patient/Doctor Flows	Application-specific logic (e.g., booking appointment, viewing records)	20%	For drafting the specific database queries and multi-step form handlers unique to the HMS.
Testing / Setup	Environment setup, virtual environment, dependency management, unit tests	10%	Useful for drafting configuration files (requirements.txt) and basic unit test stubs.

Technologies Used

Backend

- Python 3
- Flask
- SQLAlchemy
- SQLite (development)

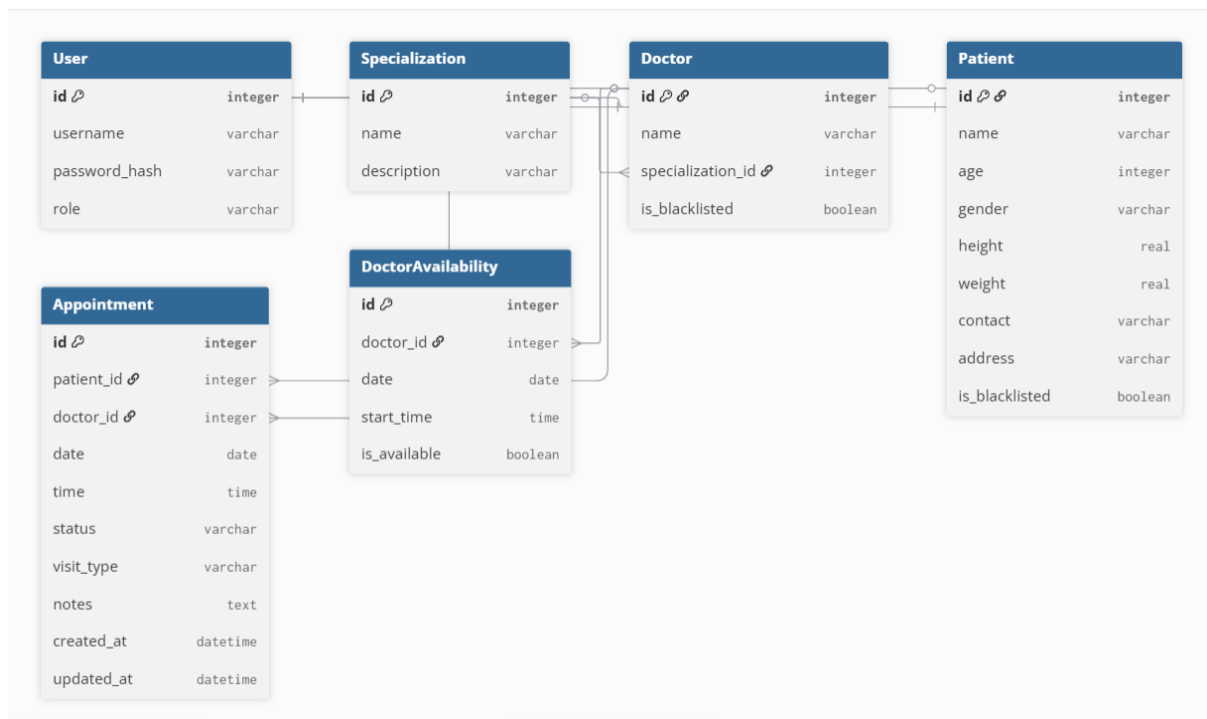
Frontend

- HTML
- CSS
- Bootstrap 5
- Jinja2 Templates

Purpose of Technologies

- Flask: Lightweight, ideal for small-to-medium applications with simple routing.
- SQLAlchemy: Ensures clean ORM mapping and easy database interactions.
- Jinja2: Enables server-side dynamic web pages.
- Bootstrap: To maintain a quick, responsive UI without custom CSS.

DB Schema Design



The project contains lightweight internal API-like routes to manage system functions:

APIs Implemented

- **User Management:**
/add_user, /edit_user/<id>, /delete_user/<id>
- **Appointments:**
/book_appointment, /view_appointments, /cancel_appointment/<id>
- **Blacklist System:**
/blacklist/<user_id>, /unblacklist/<user_id>
- **Search:**
/search_patient, /search_doctor

These routes accept form data from templates and return JSON/HTML based on context. The complete YAML API definition is included separately in the ZIP.

Architecture and Features

Project Structure (Rough)

- app.py — main entry point & route controllers
- models.py — SQLAlchemy database models

- templates/
 - admin pages
 - doctor pages
 - patient pages
 - login/signup
 - appointment pages
- static/
 - CSS, JS, images (if any)
- instance/
 - SQLite database file
- YAML File
 - Full API structure

Features Implemented

Admin Features

- Add, edit, delete users
- Blacklist / unblacklist users
- View all registered users
- View all doctor schedules
- Global search for patients & doctors

Patient Features

- Register and login
- Book appointments
- View appointment status
- Check doctor availability

Doctor Features

- View patient appointments
- Approve / reject appointment requests
- Manage availability

Additional Features

- Role-based access control
- Flash messages for feedback

- Form validation
- Simple dashboard UI

Video Presentation

Drive Link:

<https://drive.google.com/file/d/1owO9KvupGDQ8BVytF72S-CAW5BizuREI/view?usp=sharing>